Lab 5

Digital Counters Design

1. Introduction

In this lab you will use the input from the DE1-Soc pushbuttons to control digital counters. We will begin with a basic design where the state of an LED is controlled by a pushbutton. This design will be progressively extended to the point where we will utilize the pushbuttons and switches to start/stop a counter, load the counter value, and change its counting direction.

1.1 Pre-Lab Assignment

As shown in Figure 1, a clock signal is a periodic signal, constantly transitioning between logic values 0 and 1 with symmetric pulses, as shown in the timing diagram. The following properties of a clock signal can be highlighted:

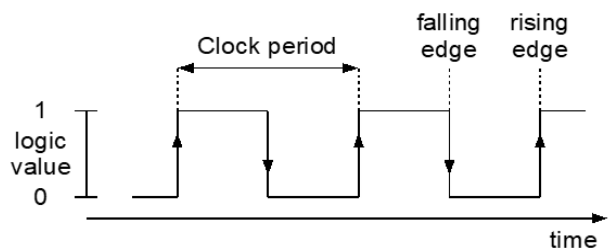

Figure 1: Clock signal

- A rising edge is a transition from a value of 0 to a value of 1.
- A falling edge is a transition from a value of 1 to a value of 0.
- The clock period is defined as the time between one rising edge to the next (or one falling edge to the next). The clock period of the DE1-SoC board is 0.02µs or 20ns or 2 x 10^{-8} second.
- The clock frequency is defined as the reciprocal of the clock period. Therefore, the clock frequency of the DE1-SoC's FPGA is 50MHz.

Clock signals are typically connected to all storage components (i.e., the flip flops) of the sequential circuits in a digital design. Their purpose is synchronizing the transition of these components into their next state. This transition happens at either the rising or falling (triggering) edge of each clock cycle.

An *n*-bit counter is a logic component storing an *n*-bit binary number that is incremented (up), or decremented (down), on every triggering edge of a clock signal. An *up free-running* (plain binary) counter is a counter that takes all possible *n*-bit values incrementally (from 0 to 2^n-1) and is reset to 0 when it reaches its max value. Its counterpart is a *limited-range* (Modulus) counter, which resets at a configured value (not the default the max value, which is 2^n-1).

- The Quartus **LPM_COUNTER IP** core is a binary counter that can be used to generate up and down counters with outputs of up to 256 bits wide. Figure 2 shows the ports (interface points) for the LPM_COUNTER IP core. The counter can be configured following the same steps used to configure the other IP Catalog library components we used in the previous lab.

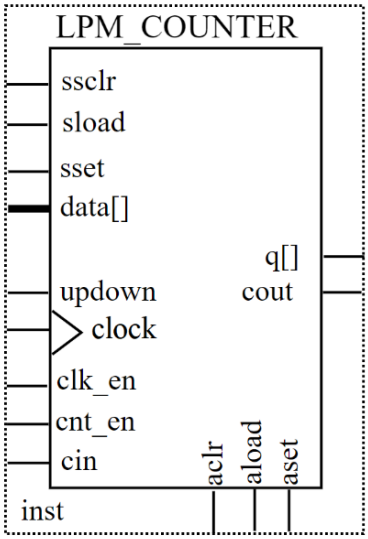

Figure 2: LPM_COUNTER Ports

Pre-Lab Reading Assignments

Read sections *2.5 Ports* and *2.6 Parameters* in the *Intel FPGA Integer Arithmetic IP Cores User Guide* available in the *Lab Resources* section on Canvas. These sections explain the functions of the counter ports and how to configure them.

Have the prelab assignment answers ready to be checked at the beginning of the lab session.

2. ON/OFF Toggling an LED with a Pushbutton

Our first goal is configuring the DE1-SoC to control the state of an LED using the KEY0 pushbutton. Every time this button is pushed, the LED toggles its state, on → off, or off → on. The DE1-SoC pushbuttons are **active low** (i.e., their output is low when pushed). If you connect a pushbutton directly to an LED, the LED will turn ON as long as the button is not pushed, and it will turn OFF as long as the button is pushed. So, how can we achieve the needed toggling behavior instead?

This behavior can be achieved with a simple 1-bit counter whose output is connected directly to the LED. A 1-bit counter is one that can take only values 0 and 1. When it reaches its maximum value 1, the counter overflows and goes back to 0. Connect the clock of this counter to the pushbutton through an inverter (as the button is active low) as shown in Figure 3. This way, pushing the button will trigger the counter with the positive edge on its clock making it count up by one.

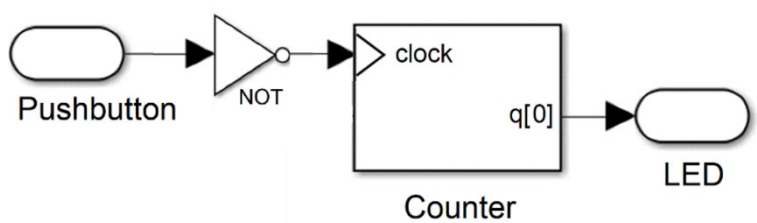

Figure 3: A counter for LED toggle

To implement the LED toggle circuit, do the following:

1. Create a new folder for a new project named **LED_toggle** on your computer. Create the new project and set the project folder to the **LED_toggle** folder and project name to **LED_toggle**.
2. Add an **LPM_COUNTER** and name it **Counter1**. Configure it as a 1-bit Up only, plain binary counter. Leave all settings to their default and make sure to check all boxes in the **Summary** window then click **Finish**.
3. Insert an instant of this counter in your schematic and name it **Counter**. Connect the output of **Counter** to an output port component, which will be mapped to an LED.

   Hint: you can type "counter" in the IP Catalog search field to find the needed LPM counter.
4. Connect the clock input of the counter to an input port component, which will be mapped to pushbutton 0.
5. **Compile** the schematic using the menu icon or using Ctrl + L. Verify there are no errors.

6. Go to **Assignments → Pin planner**.
   Connect the Pushbutton input to KEY[0] pushbutton (**PIN_AA14**).
   Connect the output to LEDR[0] (**PIN_V16**).
7. Use I/O Standards **3.3-V LVTTL** for all of them
8. Compile again.
9. Pack the **LED_toggle** schematic as a logic block named **LED_toggle.bsf** for later use. To do this, go back to your **LED_toggle** schematic and on the **File** main menu, select **Create/Update → Create Symbol Files for Current File** and save the schematic as **LED_toggle.bsf** file.

Assignment 1

Upload your **LED_toggle** design to the DE1-SoC board and test it on LED0 with KEY0 pushbutton as the input. Pushing the button should toggle the LED (change the state of the LED). Take a screenshot of your design and include it in the lab report.

3. Pushbutton-Controlled 4-bit Counter

Our next goal is to create a design with a pushbutton-controlled 4-bit counter. The output of this counter will be displayed on four LEDs. Here the design is similar to the one shown in Figure 3 but with a 4-bit counter instead of a 1-bit counter.

1. Create a new folder for a new project named **pushbutton_counter** on your computer. Create the new project and set the project folder to the **pushbutton_counter** folder and project name to **pushbutton_counter**.
2. Configure an LPM_COUNTER, and name it **FourBitCounter**, as a 4-bit Up only, plain binary counter. Leave all settings to their default and make sure to check all boxes in the **Summary** window then click **Finish**.
3. Insert an instant of this counter in your schematic and name it **PCounter**.
4. Connect the **clock** input of **PCounter** to an input port component and map it to pushbutton 0.
5. Connect the outputs of **PCounter** to four output ports component, compile, and map them to **LEDR[3..0]**.
6. Compile your project again.

Assignment 2

Upload your **pushbutton_counter** design to the DE1-SoC board and verify its counting behavior in response to pressing the pushbutton. Take a screenshot of your design and include it in the lab report.

4. Free-Running 4-bit Counter

Now we need to design a free-running 4-bit counter. It is a counter that is controlled by the board built-in clock, not by a pushbutton like the previous counter. The output of this counter will be connected to two 7-Segment displays on the DE1-SoC board. Every time the counter transitions to a new state, the 7-Segment displays will reflect the new count value.

The DE1-SoC has a default clock frequency of 50MHz. This means that synthesizing a counter on the board will make it change its state to the next count at this specific frequency. The state transitions would happen so fast that the intermediate values on the 7-Segment displays wouldn't even be observable by the human eye. We will leverage cascaded counters to reduce the counting speed.

1. Create a new folder for a new project named **free4bit_counter** on your computer. Create the new project and set the project folder to the **free4bit_counter** folder and project name to **free4bit_counter**.
2. Configure a new **LPM_COUNTER** (name it **Counter50M**) to count up to 50,000,000, which makes it overflow and reset to zero every one second. Set the counter's output number of bits to the minimum number of bits needed for a decimal value of 50,000,000 (calculate it!). It should be *Up only*, *Modulus* counter with a count modulus value set to 50,000,000. Leave everything else to their default. Check everything in the **Summary** window. Add an instant of this counter in your schematic.
3. Configure a new **LPM_COMPARE** block (name it **Compare1**) to compare the output of **Counter50M** with a constant 1. Set the compare block to be an equality checker (*a == b*) with the same number of bits as those of **Counter50M**, and ***datab*** set to a constant 1.
4. Configure a second **LPM_COUNTER** (name it **Counter15**) as a 4-bits *Up only* counter, with a ***Count Enable*** (**cnt_en**) input. This counter should be enabled with the output of **Compare1**.
5. Connect the clock inputs for both counters to the same clock input port. You can use the **CLOCK_50** (it is one of the DE1-SoC FPGA's 50MHz clock) input pin the same way you connect a switch input pin. Refer to the *PIN Assignment Numbers* slides in the *Quartus Prime Lite Reference* on Canvas.
6. Display the output value of **Counter15** on the two 7-Segment displays. To do that you will need to copy the following files from your previous lab projects to this project folder:
   - **display7_en.bsf**, **display7_en.bdf**
   - **Bin2BCD.bsf**, **Bin2BCD.bdf**

   The output from **Counter15** needs to be connected to the **Bin2BCD** module. Connect the **Bin2BCD** output to two **display7_en** modules. Your design **must not** display zero on the tens digit.
7. Compile and upload this circuit to your DE1-SoC board and test it.

Assignment 3

Test the design on the DE1-SoC FPGA. Verify that the frequence of the counter is one second and explain how this is achieved by connecting the output of **Compare1** to the **cnt_en** input of **Counter15**. How the counter behavior will change if you set ***datab*** in Compare1 to a constant different than 1 (e.g., 5). Include a screenshot of your Schematic design in your report.

Reconfigure the **Counter50M** settings so that the counter speed is doubled (i.e., changes every half a second instead of every one second). Explain your new configuration. Record a 10-second (maximum) video of testing this new counter design and upload the video with your submission.

5. Fully Controlled 4-bit Counter

The goal is to develop a counter that is controlled by pushbuttons and switches. You will start with the free-running 4-bit counter that you designed in the previous section. The overall functionality of the counter is controlled by the pushbutton keys and switches as follows:

SW[9]:	start or stop counting
SW[8]:	change the direction (i.e., increment vs. decrement)
KEY0:	load counter value from switches
SW[3..0]:	define the start value of the counter

Do not try to design all of this at once. The design would be too complex. As with any design, it is a good idea to start with small steps. The following steps guide you through the steps individually.

5.1 Start / Stop Counter

Make a copy of the free-running 4-bit counter design from the previous section, and name it **controlled_4bit_counter**. Refer to the *Quartus Prime Lite Reference* on Canvas for the steps of copying a project. In the new project, implement **SW[9]** operation to be able to start or stop the counter. The output of this switch can be used to enable **Counter15**. Display the output value of the counter on the 7-Segment displays.

Assignment 4

Validate the functionality of the **controlled_4bit_counter** on the DE1-SoC board. Add a screenshot of your new design in your report. Describe in your lab report how you achieved its start/stop functionality.

5.2 Bi-Directional Counter

Expand the design to toggle the direction of counting based on the position of **SW[8]**. If SW[8] is in the ON position, the counter must count upwards (i.e., increment its value), and if in the OFF position, the counter must count downwards (i.e., decrement its value). Double click on **Counter15** and change the counter to be an up/down counter with ***updown*** control input. Save changes for this instance of the LPM_COUNTER and use the new input to implement the bi-directional counter. If prompted to update the symbol say "Yes" → Check "*Selected symbol(s) or block(s)*" → OK. Display the output value of the counter on the 7-Segment displays.

Note: whenever you change the configuration of an LPM instant component, the connections to the component might get disconnected. So, make sure to verify the connections and fix them if needed.

Assignment 5

Validate the up/down functionality on the DE1-SoC board. Add a screenshot of your new design in your report. Describe in your lab report how you achieved the direction control functionality.

5.3 Loading the Counter with a Value

Expand the design to allow the user to set a starting value of the **Counter15** counter through the (**SW[3..0]**) switches. The **Counter15** counter should load the value from the switches only if the pushbutton **KEY0** is pushed. Otherwise, the 4-bit counter should count freely, as before.

Note that the load operation should be independent of the clock (asynchronous load). Also recall that the DE1-SoC pushbuttons are active low (i.e., they output value 1 when they are not pushed). You only need to load the value from the switches once for every push of the pushbutton . Double click and modify the **Counter15** counter configuration to allow for this "load" operation. Display the output value of the counter on the 7-Segment displays.

Note: whenever you change the configuration of an LPM instant component, the connections to the component might get disconnected. So, make sure to verify the connections and fix them if needed.

Assignment 6

Validate the loading functionality on the DE1-SoC board. Add a screenshot of your new design in your report. Describe in your lab report how you achieved the load value operation functionality.

6. Lab Submission Instructions

- The lab is intended to be implemented by each **individual** student. It is also allowed for a **maximum of two students** to work in the lab together. If you decide to work in a group of two students, **both students** must **check off** the working lab assignments with a lab instructor and only **one student** must complete the lab submission as explained in the lab assignment on Canvas.
- Write your lab report following the **report template** provided on Canvas. If you are working with a partner, write both names/IDs on the cover page. It is **prohibited** to include a partner's name on a lab report cover page if that partner does not actively participate in implementing the lab.
- The pre-lab questions prepare you for the challenges you will be presented with in the actual lab. Have their answers ready before the lab session and include these answers in the lab report together with the lab assignments.
- For each lab, submit the following on Canvas before the announced due date/time (only submissions on Canvas are accepted):
    - The lab report as a single Word or PDF document
    - Any files the assignments might ask for (e.g., a demonstration video file)
    - Submit each of the above files separately (do not upload compressed files).
- You can submit multiple attempts for this lab; however, only what you submit in the last attempt will be graded (i.e., all required reports and files must be included in this last attempt).
- **Late submission** of any lab assignment after the given due date and time will be penalized at 1 point per every late hour. No assignment submission will be accepted 48 hours after its due date/time.
- **Successful submission**: it is your responsibility to verify that you submitted successfully all the required files for the lab. First, to verify that your files were submitted, go to the assignment details page, and make sure that it says "Successfully submitted" under grade. Second, you need to validate your submission details by clicking the Submission and Rubric link on that page.
- **Deliberate plagiarism** in any assignment is an extremely serious offense that may result in failing the course. Plagiarism includes submitting answers that are not yours (e.g., copied from someone else or from the Internet). Copying materials from the course slides posted on Canvas is not considered plagiarism.